FACTORY METHOD PATTERN

Creating Objects Without Knowing Their Exact Type



ABSTRACTION



CREATION



FLEXIBILITY



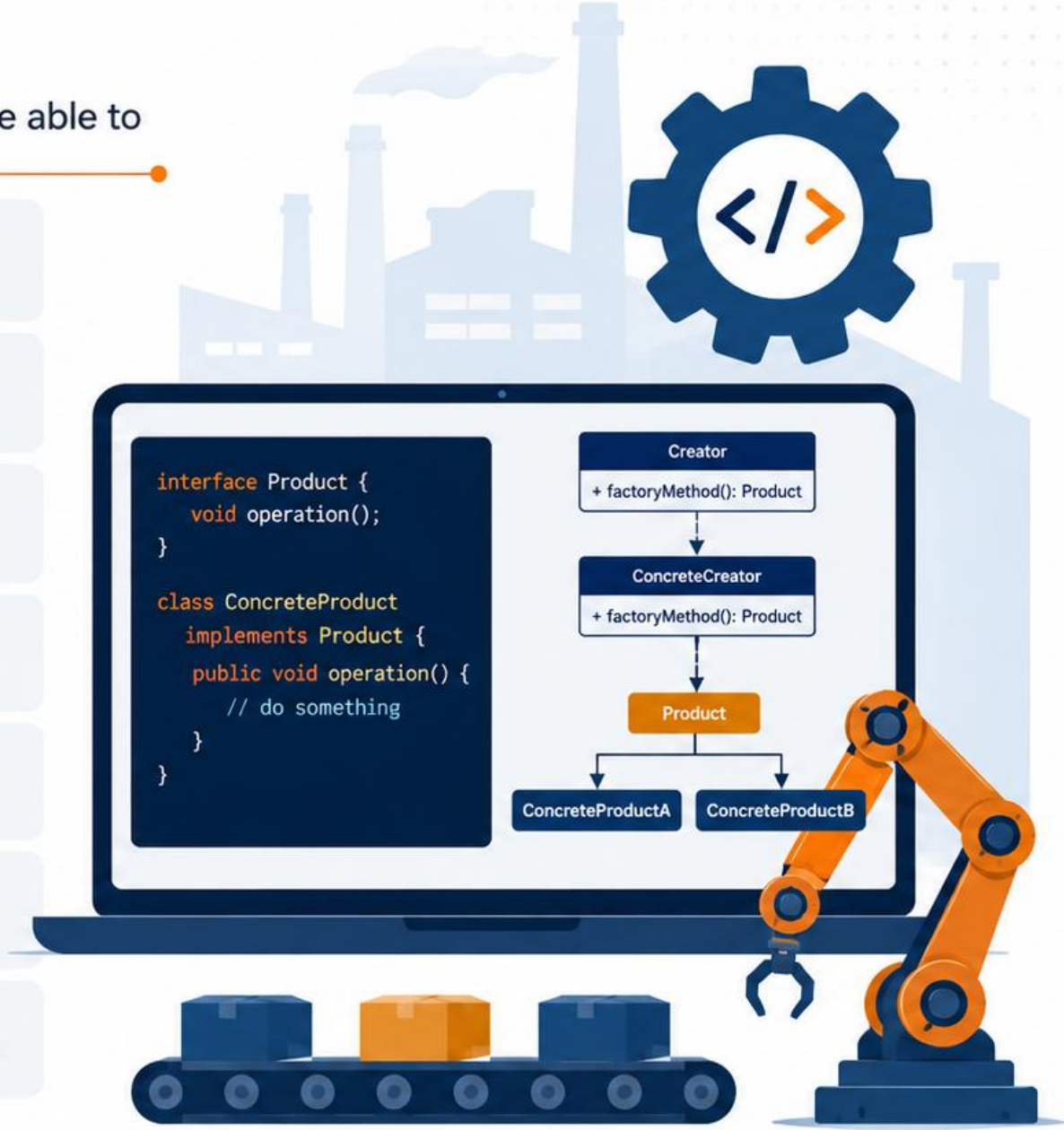
MAINTAINABILITY



Learning Outcomes

After completing the pre-class activities students should be able to

1. Explain why object creation can become a maintenance problem.
2. Identify tightly coupled code.
3. Explain the Factory Method pattern.
4. Draw the class diagram.
5. Implement Factory Method in Java.
6. Compare Simple Factory and Factory Method.
7. Apply Factory Method in a real software system.



RECAP (5 MINUTES)

Students already know

- Classes
- Objects
- Inheritance
- Interfaces
- Polymorphism

QUESTION

Which OOP principle allows us to write the following?

```
Vehicle v = new Car();
```

instead of

```
Car c   = new Car();
```

PURPOSE

Refresh polymorphism.

KEY TAKEAWAY

Polymorphism lets us write flexible and reusable code.
We program to an abstraction (Vehicle),
but the actual object (Car or Bike) decides the behavior at runtime.

POLYMORPHISM IN DETAIL

Polymorphism (many forms) allows objects of different classes to be treated as objects of a common super class (or interface). The actual method that runs is decided at runtime based on the actual object.

How it works

Reference type (superclass/interface) → Holds an object of a subclass → Method is called on the reference → Runtime decides which version to execute (Method Overriding)

Example – Vehicle

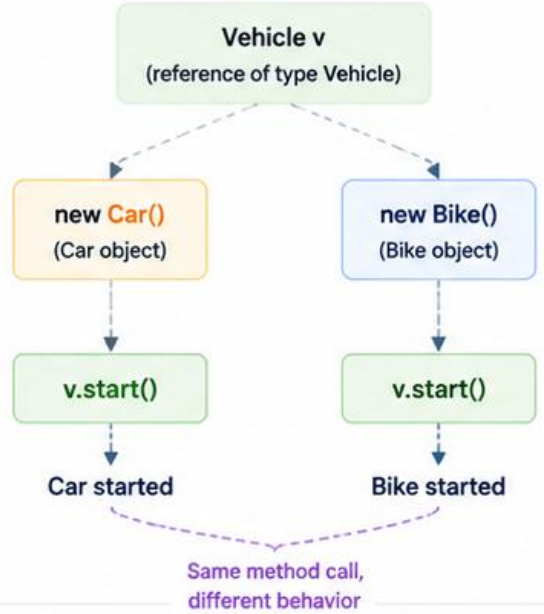
```
interface Vehicle {
   void start();
}

class Car implements Vehicle {
   public void start() {
      System.out.println("Car started");
   }
}

class Bike implements Vehicle {
   public void start() {
      System.out.println("Bike started");
   }
}

// Polymorphism in action
Vehicle v1 = new Car();  // Car object
Vehicle v2 = new Bike(); // Bike object

v1.start();   // Output: Car started
v2.start();   // Output: Bike started
```



Remember: Polymorphism + Inheritance/Interfaces + Method Overriding = Runtime flexibility

OPENING CASE STUDY

Food Delivery System

CASE

A food delivery system supports the following food types.


Pizza


Burger


Pasta

NEW REQUIREMENT

Management decides to introduce new food types.


Sushi


Noodles


Kottu


Rice & Curry

SCENARIO

The system must be updated to support the new food types.

The current implementation is shown here.



CURRENT IMPLEMENTATION

```
Food getFood(String type) {
    Food food = null;
    if(type.equals("Pizza"))
        food = new Pizza();
    else if(type.equals("Burger"))
        food = new Burger();
    else if(type.equals("Pasta"))
        food = new Pasta();
    // more else if blocks will be added
    return food;
}
```

WHAT HAPPENS WHEN...

- We add Sushi, Noodles, Kottu, Rice & Curry?
- More food types will be added in the future?
- Many developers work on this code?
- We want to test each food type easily?

QUESTION

Identify **THREE** software engineering problems in the current implementation.



EXPECTED PROBLEMS (Any Three)

Tight Coupling
Client code is tightly coupled with concrete classes (Pizza, Burger, Pasta, ...).

Difficult Maintenance
Every time a new food type is added, this method must be modified.

Violates Open/Closed Principle
The code is not open for extension but must be modified.

Many if-else Statements
The method grows with many conditional blocks, making it messy and error-prone.

Difficult Testing
Hard to isolate and test object creation logic for each food type.

GOAL: Design the system to add new food types without modifying existing code.

Think. Discuss. Solve.

WHY IS THIS A PROBLEM?

The current approach does not scale!

TODAY – SIMPLE CASE

Client

</> if / else if / else...


Pizza


Burger


Pasta

Works fine with a few types.
Easy to understand and manage.

AFTER ONE YEAR – GROWING COMPLEXITY

Client

</> if / else if / else...


Pizza


Burger


Pasta


Sushi


Kottu


Rice & Curry


Noodles

and many more...

Every new type requires code changes.
The if-else chain becomes longer and riskier.

Problems Start Here!

- ✗ More lines of code
- ✗ Hard to maintain
- ✗ Easy to introduce bugs
- ✗ Violates OCP
- ✗ Hard to test

QUESTION | What happens when there are 50 food types?



As the number of types grows, the code becomes harder to change, test, and maintain.
We need a better way to create objects!

Common Properties

Explained with a Food Delivery System

Different objects

The system may need different objects at different times (e.g., Restaurant, Customer, Rider, Payment Method).

Same actions

Even though the objects are different, they perform similar actions (e.g., processOrder(), confirm(), deliver()).

Solution:
Delegate the object creation for a Factory class.

Food Delivery System Example

Problem: Creating objects directly



Client
(FoodDeliveryApp)

```
Restaurant r = new Restaurant();
Customer c = new Customer();
Rider rd = new Rider();
CreditCardPayment p = new CreditCardPayment();
FoodOrder o = new FoodOrder();
// Tight coupling with concrete classes
```

- Client knows concrete classes
- Hard to change or extend
- Violation of Open/Closed Principle
- Difficult to test

Solution: Use a Factory to create objects



Client
(FoodDeliveryApp)

FoodDeliveryFactory

+ createRestaurant()
+ createCustomer()
+ createRider()
+ createPayment(method)
+ createOrder()

- Restaurant
- Customer
- Rider
- Payment Method (Card / Cash / Wallet)
- Food Order

- Client depends on factory, not concrete classes
- Easy to extend new types
- Follows Open/Closed Principle
- Easier testing and maintenance

Key Takeaway

Factory Pattern centralizes object creation, reduces coupling, and makes the system flexible and easy to extend.

Loose Coupling | Open for Extension | Easy Testing | Maintainable | Reusable

Same Type, Different Usages

One Type, Many Roles!

Home-made Example



In the same way, in software we create objects of the **same type** (e.g., Food) and use them in **many different ways**.

Key Takeaway

A single type (**Spoon**)
can have many **responsibilities**
depending on how and where it is used.

Software Analogy

One class or interface (e.g., **Food**)
can have many **implementations**
(**Pizza**, **Burger**, **Pasta**, **Sushi**, ...)
and be used in many parts of the system
in different ways.

QUESTION | What happens when there are **50 food types**?



Think About It!

How does the current implementation handle growth?
What are the risks?

Challenge: The current approach works for a few types, but does not scale and becomes hard to maintain.
We need a better way to create objects!

THINK LIKE A FACTORY

A real automobile factory.

DISCUSSION

Does a customer build the car?
No.

The customer only asks
“I need an SUV.”

The factory creates it.

Programming should work the same way.



Just like a factory takes raw materials and produces the exact car you need, a **Factory Method** creates the exact object your program needs.

Factories hide the creation process and give you the right product.
You just request. The factory delivers.

EVERYDAY EXAMPLES

Students identify products created by factories

Factories create different products that belong to the same category. They decide what to make – not the customer.

1 ATM



- Savings Account
- Current Account
- Fixed Deposit

2 HOSPITAL



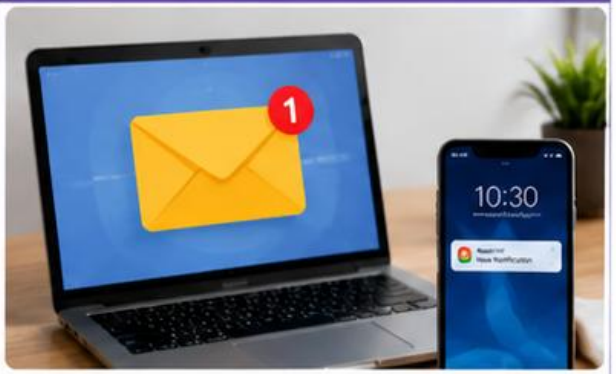
- Doctor
- Dentist
- Surgeon

3 RIDE SHARING



- Car
- Bike
- Van

4 NOTIFICATION SYSTEM



- Email
- SMS
- Push Notification

MINI QUESTION

Who decides which object should be created?

- The customer? ✗ No
- The factory? ✓ Yes
- The machine? ✗ No

Key Idea: A factory (or factory method) decides which object to create. The client only requests – the factory provides the right object.

CODE SMELL

A warning sign in your code

Code smell refers to any characteristic in the code that may indicate a **deeper problem in design**.

EXAMPLE: Object Creation Using if-else

```
Food createFood(String type) {
  if(type.equals("Pizza")) {
    return new Pizza();
  }
  else if(type.equals("Burger")) {
    return new Burger();
  }
  else if(type.equals("Pasta")) {
    return new Pasta();
  }
  // ... many more else if blocks
  // for Sushi, Noodles, Kottu, Rice & Curry,...
  else {
    throw new IllegalArgumentException("Invalid type");
  }
}
```

Repeated object creation is scattered throughout the code.

PROBLEMS WITH THIS APPROACH

1. Repeated Object Creation

The client class is responsible for creating objects. The same pattern is repeated in many places.

2. Large switch / if-else Statements

As the number of types grows, the method becomes long and hard to read.

3. Frequent Modifications

Every time a new type is added, this method must be modified. Risk of introducing bugs is high.

4. Hard to Extend

Adding new types requires changing existing code, making the system less flexible and hard to maintain.

CONSEQUENCE

The code violates good design principles, becomes **tightly coupled** and is difficult to maintain, test, and extend.

IMPACT AS THE SYSTEM GROWS

- Slower development
- More bugs
- Harder to test
- Higher maintenance cost

QUESTION

What principle is **violated** by the above approach?

ANSWER:

Open/Closed Principle

The system should be **open for extension** (new types) but **closed for modification** (existing code).

KEY TAKEAWAY: This code smells! We need a better way to create objects without modifying existing code.

FACTORY METHOD DEFINITION

A creational design pattern

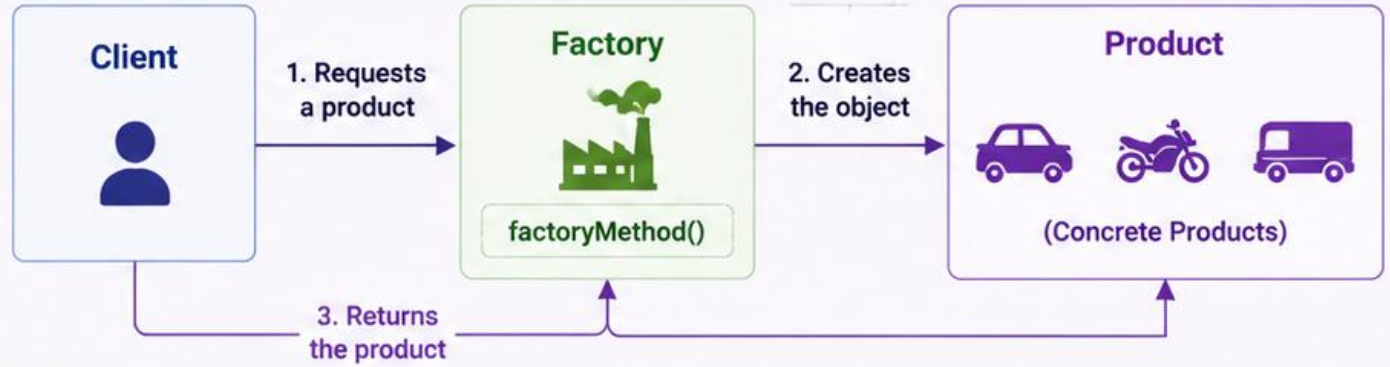
SIMPLE DEFINITION

Factory Method defines an **interface** for **creating** objects, but allows **subclasses** to decide which object to create.

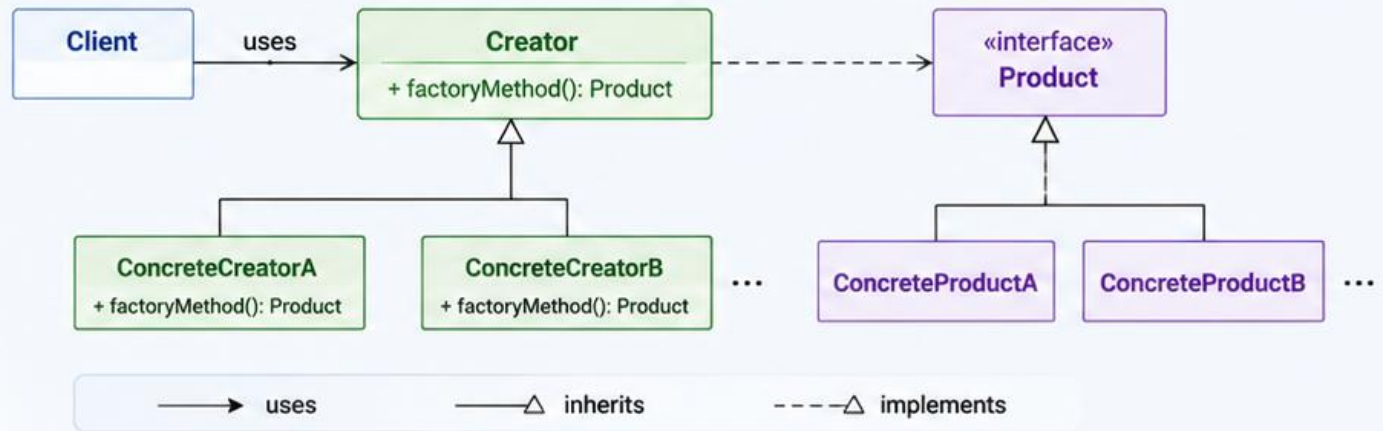
EXPLAINED IN PLAIN ENGLISH

1. The client asks for a product.
2. The factory decides which object to create.
3. The client gets the product.
4. The client never calls new directly.

HOW IT WORKS



CLASS DIAGRAM (STRUCTURE)



KEY TAKEAWAY

Factory Method lets the factory handle object creation.
The client just asks for what it needs.

Example: When you order a pizza, you don't make it.
The restaurant (factory) decides how to make it.

FACTORY METHOD – PARTICIPANTS

PRODUCT





CONCRETE PRODUCT





CREATOR





CONCRETE CREATOR





CLIENT



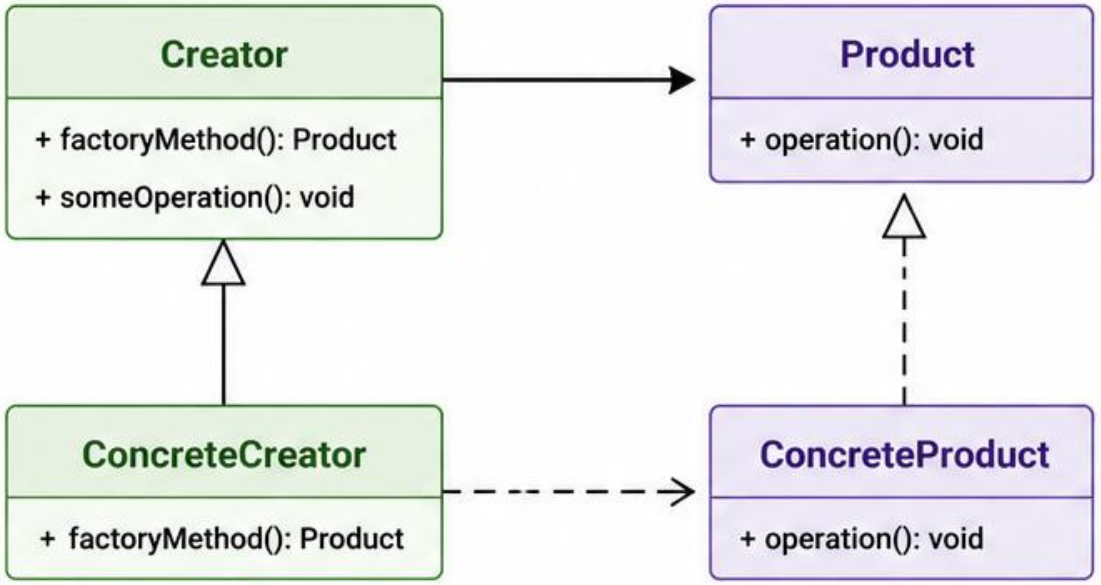


FACTORY METHOD – UML DIAGRAM (GoF)

Animating one relationship at a time

Association
Dependency
Inheritance

THE STANDARD GoF UML DIAGRAM



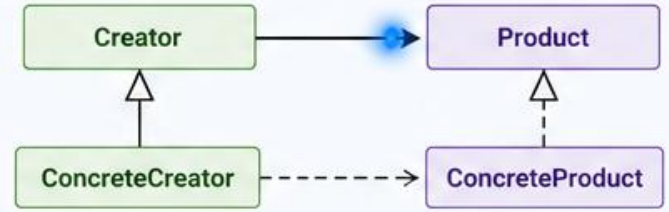
PARTICIPANTS

- **Product** – Common interface for objects.
- **Concrete Product** – Actual objects created by the factory.
- **Creator** – Declares the factory method that returns a Product.
- **Concrete Creator** – Overrides the factory method to return a Concrete Product.
- **Client** – Uses products via the Creator and Product interfaces.

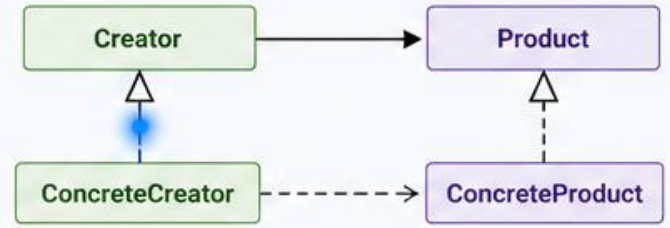
KEY IDEA

The client works with interfaces. The actual object is decided by the subclass (factory).

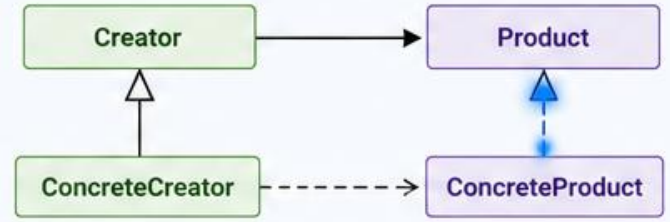
1 **Creator uses Product** (Association)



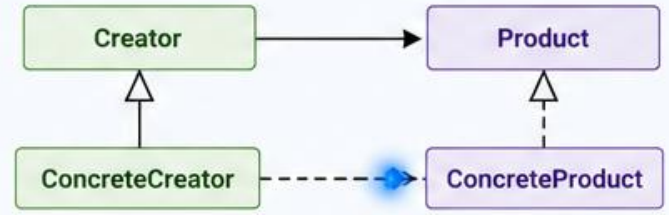
2 **ConcreteCreator overrides factoryMethod() of Creator** (Inheritance)



3 **Product is implemented by ConcreteProduct** (Inheritance)



4 **ConcreteCreator depends on ConcreteProduct** (Dependency)



5 **All relationships**



FACTORY METHOD – STEP-BY-STEP EXECUTION

Understanding the Runtime Flow

This animation shows how a request flows from the Client to the Product and back during runtime.

1 CLIENT



The client requests a product through the **Creator** (Factory) interface.

2 FACTORY (CREATOR)



The factory receives the request and calls the **factoryMethod()** to create the product.

3 CONCRETE FACTORY



The concrete factory overrides the **factoryMethod()** and creates the appropriate concrete product.

4 PRODUCT



The concrete product object is created. It is ready to be returned to the client.

5 RETURN OBJECT



The factory returns the created product object back to the client.

6 USE OBJECT

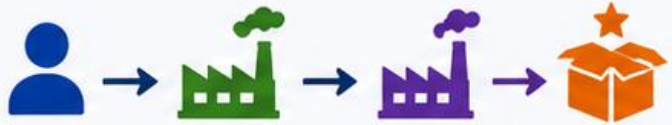


The client uses the product object through the Product interface to perform operations.

REQUEST SENT → REQUEST RECEIVED → OBJECT CREATED → PRODUCT READY → OBJECT RETURNED → OBJECT USED

KEY TAKEAWAY

The client never creates objects directly using new.
It relies on the factory to create and return the right product.



LEGEND

- Request Flow
- Creation Flow
- Return Flow
- Usage Flow

FACTORY METHOD PATTERN – JAVA EXAMPLE

Example Domain: **Payment System**

The client asks for a Payment.
The factory decides which payment object to create.

1. DOMAIN OVERVIEW

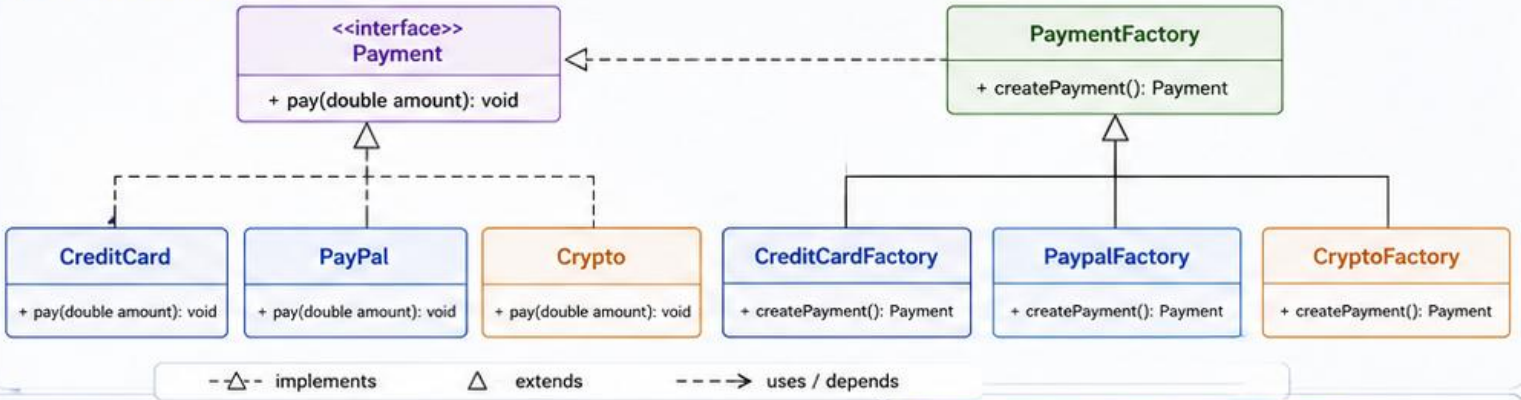
Products (Payment) → **Factories** → **Client**

- CreditCard
- PayPal
- Crypto

- PaymentFactory
- CreditCardFactory
- PaypalFactory
- CryptoFactory

PaymentSystem (Main Application)

2. CLASS DIAGRAM (UML)



3. JAVA CODE (SIMPLIFIED)

```java
// Product Interface
public interface Payment {
    void pay(double amount);
}
// Concrete Products
public class CreditCard implements Payment {
   public void pay(double amount) {
      System.out.println("Paid " + amount
          + " using Credit Card");
}
public class PayPal implements Payment {
   public void pay(double amount) {
      System.out.println("Paid " + amount
          + " using PayPal");
}
public class Crypto implements Payment {
   public void pay(double amount) {
      System.out.println("Paid " + amount
          + " using Crypto");
}
```

```java
// Factory (Creator)
public abstract class PaymentFactory {
    public abstract Payment createPayment();
}

// Concrete Factories
public class CreditCardFactory extends PaymentFactory {
   public Payment createPayment() {
      return new CreditCard();
}

public class PaypalFactory extends PaymentFactory {
   public Payment createPayment() {
      return new PayPal();
}

public class CryptoFactory extends PaymentFactory {
   public Payment createPayment() {
      return new Crypto();
}
```

```java
// Client
public class PaymentSystem {
   public static void main(String[] args) {
      PaymentFactory factory;
      Payment payment;

      factory = new CreditCardFactory();
      payment = factory.createPayment();
      payment.pay(1000.00);

      factory = new PaypalFactory();
      payment = factory.createPayment();
      payment.pay(250.50);

      factory = new CryptoFactory();
      payment = factory.createPayment();
      payment.pay(0.005);
   }
}
```

4. RUNTIME FLOW (STEP-BY-STEP)

1. **Client** (PaymentSystem) requests a Payment.
2. **PaymentFactory (Creator)** receives the request.
3. **Concrete Factory** creates the appropriate Payment object.
4. **Product** (Payment) object is created and returned.
5. **Client** uses the Payment object to perform the payment.

KEY TAKEAWAY

The client **never** creates objects directly using **new**.
It relies on **factories** to create and return the correct **Payment** object.

Example Output

```
Paid 1000.0 using Credit Card
Paid 250.5 using PayPal
Paid 0.005 using Crypto
```

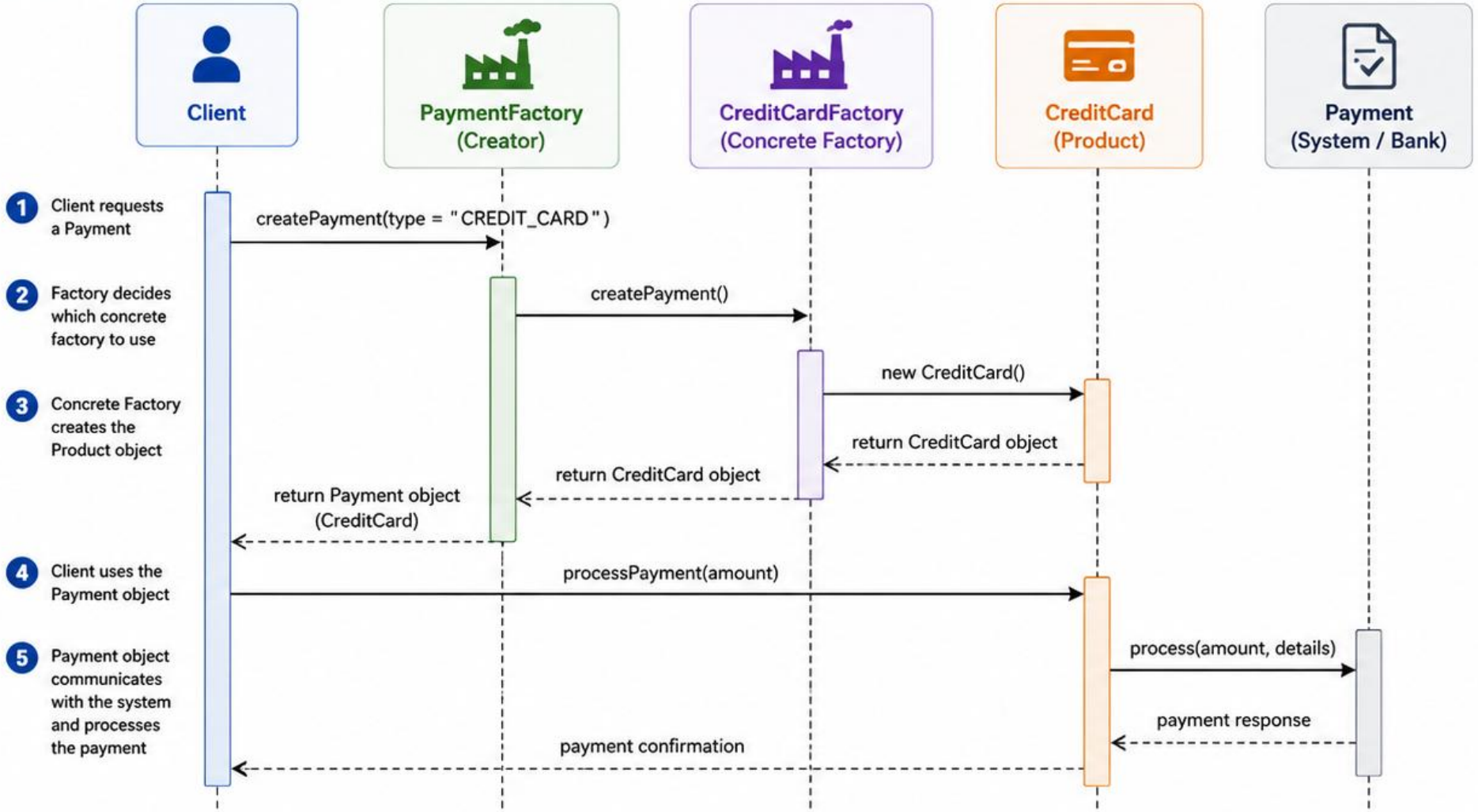
Why Factory Method?

- Decouples client from concrete classes
- Easy to add new payment methods
- Follows Open/Closed Principle

FACTORY METHOD PATTERN – SEQUENCE DIAGRAM

Example: Payment System (Credit Card Payment)

This sequence shows the runtime flow of creating a CreditCard object using Factory Method and then processing the payment.



STEP-BY-STEP EXPLANATION

1. Client (PaymentSystem) requests a payment for type CREDIT_CARD from PaymentFactory.
2. PaymentFactory decides which concrete factory to use and delegates the creation to CreditCardFactory.
3. CreditCardFactory creates a CreditCard object and returns it back through the chain.
4. Client receives the Payment object (CreditCard) and calls processPayment() on it.
5. CreditCard object processes the payment by communicating with the Payment System / Bank and returns the result.

LEGEND

- Synchronous Call
- Return
- Activation / Execution

KEY TAKEAWAY

The client never creates objects directly using new.
It relies on factories (PaymentFactory → CreditCardFactory) to create and return the correct Payment object.

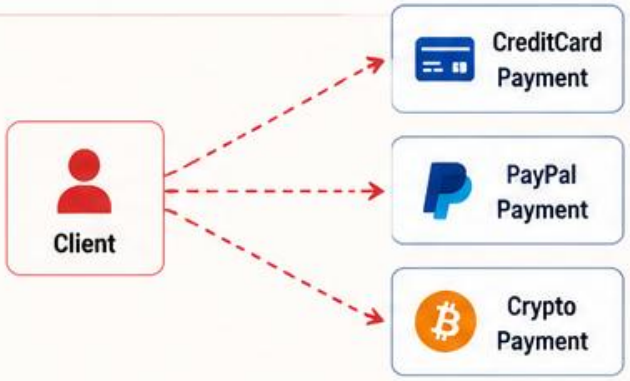
BENEFITS OF FACTORY METHOD PATTERN

Without Factory vs With Factory

WITHOUT FACTORY

Client Code

```
public void pay(String type, double amount) {
    Payment payment;
    if (type.equals("CREDIT_CARD")) {
        payment = new CreditCardPayment();
    } else if (type.equals("PAYPAL")) {
        payment = new PayPalPayment();
    } else if (type.equals("CRYPTO")) {
        payment = new CryptoPayment();
    } else {
        throw new IllegalArgumentException("Invalid type");
    }
    payment.processPayment(amount);
}
```



Problems

Large if-else

```
if (...) {
else if (...) { ... }
else { ... }
```

Tight coupling



Client depends on concrete classes

Difficult maintenance



Change in one place affects client code

Difficult testing



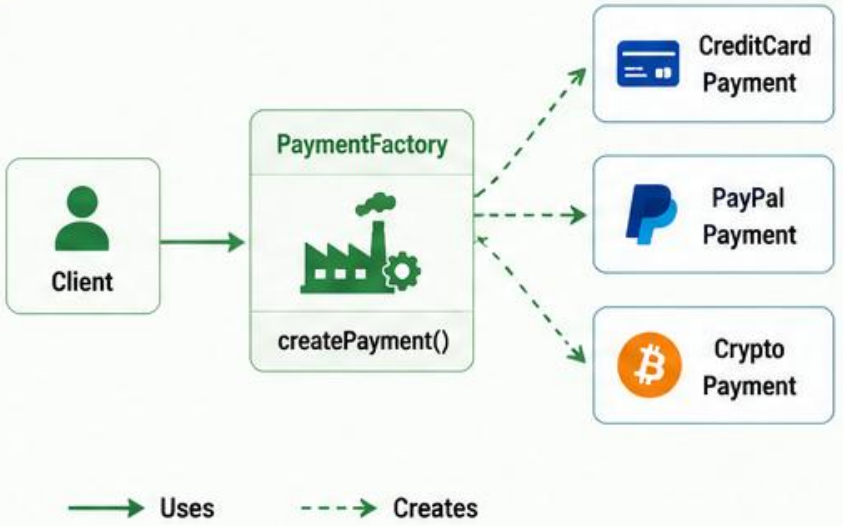
Hard to mock or replace dependencies

VS

WITH FACTORY

Client Code

```
public void pay(String type, double amount) {
    PaymentFactory factory = PaymentFactory.getFactory(type);
    Payment payment = factory.createPayment();
    payment.processPayment(amount);
}
```



Benefits

Loose coupling



Client depends on abstractions, not concrete classes

Easy extension



Add new payment types without changing client code

Better testing



Easily mock factories and test in isolation

Cleaner code



No if-else, simple and readable

WITHOUT FACTORY

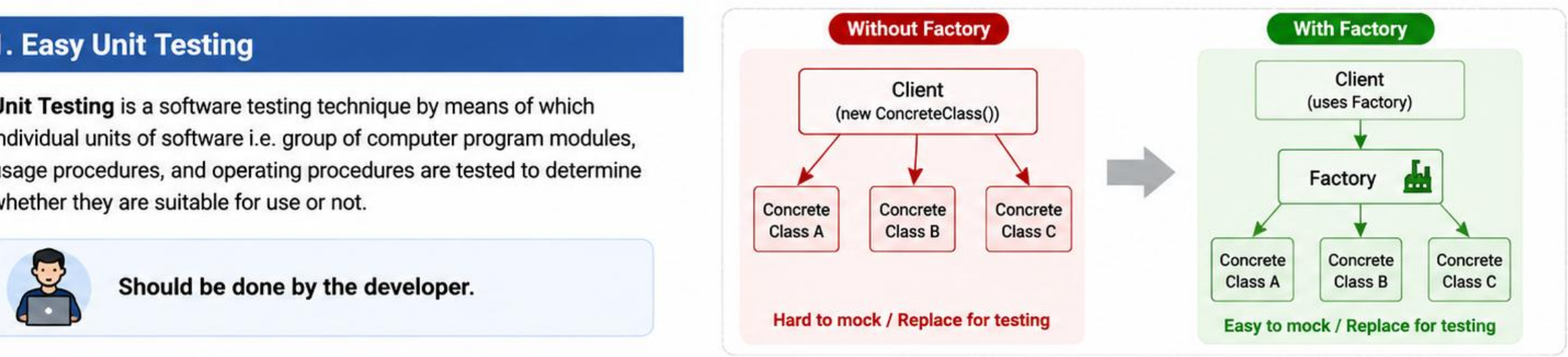
WITH FACTORY

Advantages of Factory Pattern

1. Easy Unit Testing

Unit Testing is a software testing technique by means of which individual units of software i.e. group of computer program modules, usage procedures, and operating procedures are tested to determine whether they are suitable for use or not.

Should be done by the developer.



2. Increase of Code Usability

Objects are created in a centralized way. Reusable creation logic can be used by multiple parts of the application.



3. Improve Code Readability

Clients don't need to know the details of object creation. Code becomes cleaner and easier to understand.

```
// Instead of this
ConcreteClass obj = new ConcreteClass();

// We have this
ConcreteClass obj = Factory.create();
```

4. Code Redundancy is Reduced

If multiple clients want to initiate the same set of classes, using a factory eliminates duplicate object creation code.



5. Other Benefits

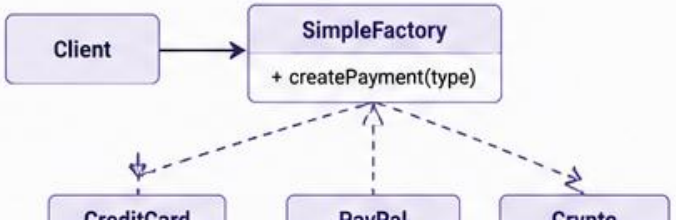
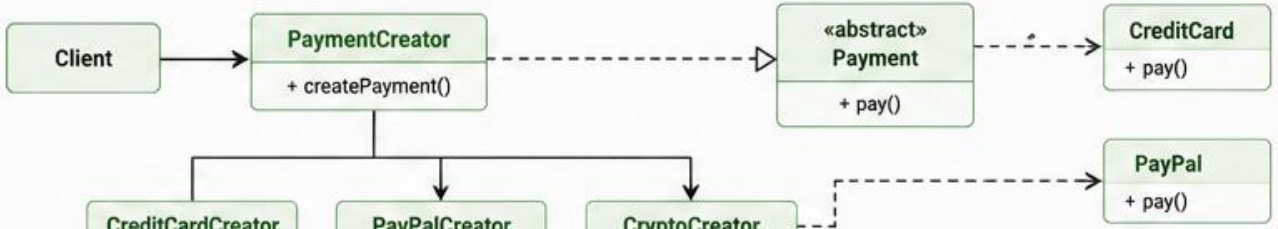
- **Code Extensibility**
  - Easy to add new types without changing existing client code.
- **Code Transparency**
  - Separation of concerns makes the system more maintainable and transparent.

Key Takeaway

Factory Pattern centralizes object creation, makes testing easier, improves code quality, reduces redundancy, and supports a flexible and maintainable system.

FACTORY VS SIMPLE FACTORY

Understand the Key Differences

	SIMPLE FACTORY	VS	FACTORY METHOD
1 Structure	**One factory class** All object creation logic is in a single class.		**Factory hierarchy** Uses a base Creator and multiple Concrete Creators (subclasses).
2 Creation Logic	**Uses if-else** The factory uses if-else or switch to decide which object to create. `if(type.equals("CREDIT_CARD")) { return new CreditCard(); } else if(type.equals("PAYPAL")) { return new PayPal(); } else if(type.equals("CRYPTO")) { return new Crypto(); }`		**Uses inheritance** Subclasses decide which object to create by overriding the factory method. `// In Concrete Creator @Override public Payment createPayment() { return new CreditCard(); }`
3 Extensibility	**Harder to extend** Adding a new product type requires modifying the factory class (violates Open/Closed Principle).		**Easy to extend** Add a new product by creating a new Concrete Creator (no changes to existing code).
4 Flexibility	**Less flexible** Tightly coupled to concrete classes and creation logic.		**Highly flexible** Loosely coupled – depends on abstractions, not concrete classes.
5 Class Diagram			

When Should We Use Factory Method?

Use Factory Method when you want flexibility, scalability and decoupling.

USE WHEN

Object type changes
The exact type of object to create may vary at runtime.

Future extensions expected
New types can be added without modifying existing client code.

Client should not know implementation
Hide concrete classes and create objects via an interface.

Multiple implementations exist
Different implementations of the same interface are available.

FACTORY METHOD

EXAMPLES

1


Payment Gateway
Different payment providers (Stripe, PayPal, Razorpay) implement the same interface.

2


Database Drivers
Different databases (MySQL, PostgreSQL, Oracle) provide their own driver implementations.

3

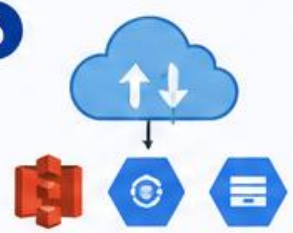

Vehicle Manufacturing
Different vehicles (Car, Truck, Bike) created by different manufacturers.

4


Document Editors
Create different documents (Word, PDF, Excel) through specific editor implementations.

5


Game Characters
Different characters (Warrior, Mage, Archer) created with different abilities.

6


Cloud Storage
Different cloud providers (AWS, Azure, Google Cloud) provide storage implementations.

Advantages of Factory Method Pattern

Advantage	Explanation
Loose Coupling	Client doesn't know concrete class.
Open for Extension	New products can be added easily without modifying existing code.
Easy Testing	Products can be mocked or stubbed for unit testing.
Maintainable	Changes are isolated to specific classes.
Reusable	Object creation logic can be reused across the application.

Factory Method promotes **flexibility**, **scalability** and **clean design** by relying on inheritance and polymorphism.

Disadvantages of Factory Method Pattern

Disadvantage	Explanation
More Classes	Requires creating creator subclasses for each product type.
Higher Complexity	Increases the structure and makes the code more complex.
Extra Abstraction	Adds additional abstraction which may be unnecessary in some cases.
Small Projects May Not Need It	For simple applications, the pattern may add unnecessary overhead.

Use Factory Method when you expect **new product types** to be added or when you need loose coupling.

Reflection Questions

01 Why is using new everywhere considered bad practice?

02 Which SOLID principle does Factory Method support?

03 How is Factory Method different from Simple Factory?

04 Why does the client depend only on interfaces?

05 Give two real-world systems where Factory Method can be applied.



Factory Method Design Pattern – Summary

Define an interface for creating an object, but let subclasses decide which class to instantiate.

1. PURPOSE

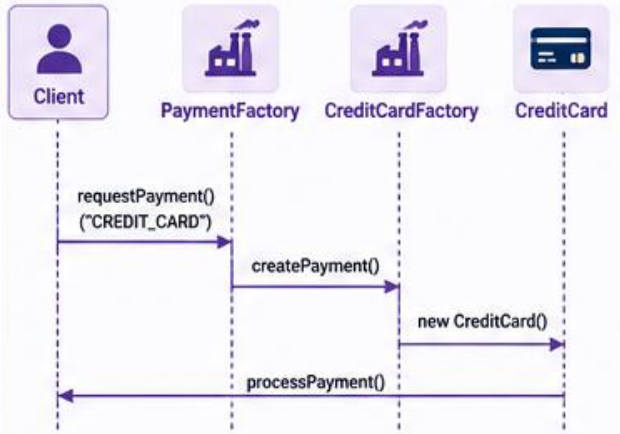
Factory Method lets a class defer instantiation to its subclasses.

Promotes loose coupling and makes the system open for extension but closed for modification.

2. REAL-WORLD EXAMPLE – PAYMENT SYSTEM



3. SEQUENCE DIAGRAM (RUNTIME FLOW)



Shows how objects are created and used at runtime. Client depends on abstractions, not concrete classes.

4. BENEFITS – AT A GLANCE

Without Factory	With Factory
Large if-else or switch statements	Loose coupling between client and products
Tight coupling to concrete classes	Easy to extend with new products
Difficult maintenance	Better testing with mock objects
Difficult testing	Cleaner code and better readability

5. FACTORY METHOD vs SIMPLE FACTORY

Aspect	Simple Factory	Factory Method
Structure	One factory class	Factory hierarchy
How it works	Uses if-else or switch	Uses inheritance and polymorphism
Extensibility	Harder to extend	Easy to extend
Flexibility	Less flexible	Highly flexible
Best for	Small and stable systems	Systems expecting change

6. WHEN TO USE FACTORY METHOD?

Use When:

- The type of object to create may change.
- Future extensions are expected.
- The client should not know implementation details.
- Multiple implementations exist for the same interface.

Common Examples:

Payment Gateway · Database Drivers · Vehicle Manufacturing · Document Editors · Game Characters · Cloud Storage

7. REFLECTION QUESTIONS

01. Why is using new everywhere considered bad practice?
02. Which SOLID principle does Factory Method support?
03. How is Factory Method different from Simple Factory?
04. Why does the client depend only on interfaces?
05. Give two real-world systems where Factory Method can be applied.